

Prompt Engineering and LLM Applications: Context, Structure, Tools and Evaluation

Unit 4 of the Generative AI and LLM crash course

GenAI crash course

2026-10-09

Contents

Session map	2
1. Terms: the term tree	3
2. Remember	4
3. Understand: the derivation	5
3.1 Tier 1 (easy): intuition and a picture	5
3.2 Tier 2 (medium): the definitions made precise	5
3.2.1 Templates	5
3.2.2 Conversation cost: the model has no memory	7
3.2.3 Chain-of-thought	7
3.2.4 Structured output: validate, repair, retry	7
3.2.5 Tool calling	8
3.3 Tier 3 (medium-hard): the main derivations	9
3.3.1 Self-consistency: when does voting help?	9
3.3.2 Comparing two prompts: paired data and sample size	10
3.3.3 The winner's curse	11
3.4 Tier 4 (hard): robustness and risk	11
3.4.1 Long contexts	11
3.4.2 Constrained output	11
3.4.3 Prompt injection (preview)	12
3.5 Tier 5 (hardest): open questions	12
4. Apply	12

4.1 Worked problems	12
4.2 Code and figures	12
5. Analyze	13
5.1 Limits	13
5.2 Which term dominates	13
5.3 Sensitivity	14
5.4 Failure modes	14
5.5 Constructed versus measured	14
6. Evaluate	14
7. Create	15
8. Visualization lesson	15
9. Problem set	16
10. Retrieval practice and recap	16
11. Where next	16

Session map

Outcome of this session

After this session you can:

1. explain a prompt as the *prefix an LLM continues* (Unit 3), and build zero-shot, few-shot, role and chain-of-thought prompts for a task;
2. budget a **context window**, and say what is silently lost when a conversation gets too long;
3. make a model's output **machine-readable**: schema, validation, repair and retry, and know what each can and cannot fix;
4. build a **tool-calling** loop with validated arguments, and say what can go wrong;
5. **evaluate prompts honestly**: derive majority-vote accuracy, run a paired comparison with a confidence interval, and say how many test items you need.

- **Prerequisites:** Unit 3: next-token prediction, decoding, temperature and the context window. Unit 2: the Transformer, in particular that attention cost grows with context length. Basic probability (binomial distribution) for Section 3.3.
- **Difficulty ladder used:** easy → medium → medium-hard → hard (Sections 3.1 to 3.4); the hardest tier is posed as open questions in Section 3.5.

- **Deliverables:** this document (.tex, .pdf, .docx), 5 figures, the module codes/`llm/prompting_tools/` (templates, context budgeting, JSON validation and repair, tool calling with a mock model, majority-vote and bootstrap statistics; 9 tests), the *Context window budget* and *Majority vote* widgets, and a Colab notebook for the three syllabus projects with a real small model.
- **Source anchors:** Brown et al. 2020, arXiv:2005.14165 (few-shot prompting); Wei et al. 2022, arXiv:2201.11903 (chain-of-thought; the quoted result, a PaLM 540B model with eight exemplars reaching state-of-the-art accuracy on GSM8K, is from its introduction); Kojima et al. 2022, arXiv:2205.11916 (the “Let’s think step by step” zero-shot prompt); Wang et al. 2022, arXiv:2203.11171 (self-consistency, “sample-and-marginalize”, majority vote in its Table 1); Liu et al. 2023, arXiv:2307.03172 (U-shaped use of long contexts). All IDs and quoted statements were checked in the PDFs on the day of writing.
- **An honest limit of this session.** There is no language model and no API key on the machine that built this course. Everything here that is *measured* uses either exact mathematics, a clearly labelled **simulation**, or a **mock model** that only tests the plumbing (it understands nothing). Statements about how real models behave are cited to the papers above and were not re-measured. The practicals with a real model are in a Colab notebook that was **not executed** while writing this document.

1. Terms: the term tree

Term	Symbol	One-line meaning	Where it appears	New / known
prompt		the text (prefix) the model is asked to continue	3.1	new
system / user / assistant / tool message		the roles in a chat request	3.1	new
zero-shot, few-shot prompting		task description only / plus worked examples	3.1	new
in-context learning		the model adapts to examples in the prompt, with no weight update	3.1	new
chain-of-thought (CoT)		the model writes intermediate steps before the answer	3.2	new
prompt tem- plate		a fixed prompt with placeholders filled in per request	3.2	new
structured output		output that must follow a schema such as JSON	3.2	new

Term	Symbol	One-line meaning	Where it appears	New / known
function / tool calling		the model asks your code to run a function	3.2	new
context window manage- ment		deciding what text to keep when the history exceeds the limit	3.2	new
self- consistency		sample several answers and take the majority	3.3	new
paired compari- son		compare two prompts on the same test items	3.3	new
confidence interval	CI	a range of plausible values for the true effect	3.3	new
winner's curse		significant results overstate the true effect	3.3	new
prompt injection		text in the input that tries to override your instructions	3.4	new

Terms *used but not taught here*: retrieval and chunking (Unit 5), fine-tuning (Unit 6), agents (Unit 8), the full treatment of prompt injection and evaluation (Unit 9).

2. Remember

Prompt and chat messages

A **prompt** is the text a model is given to continue. Chat APIs structure it as a list of **messages** with roles: **system** (standing instructions), **user** (the request), **assistant** (earlier model replies) and **tool** (results of function calls). The model is *stateless*: on every call you send the whole conversation again.

Zero-shot, few-shot, chain-of-thought

Zero-shot: instructions only. **Few-shot**: instructions plus a few input-output examples (Brown et al. 2020); the model adapts from the examples in the prompt, which is called **in-context learning**, with no change to its weights. **Chain-of-thought**: the prompt (examples or the phrase “Let’s think step by step”) makes the model write intermediate reasoning before the final answer (Wei et al. 2022; Kojima et al. 2022).

Structured output and tool calling

Structured output: the reply must be, for example, JSON that matches a **schema** (required keys, types, allowed values). **Tool calling**: the model replies with a request such as `{"tool": "calculator", "arguments": {"expression": "23*(4+6)"}}`; *your code* validates and runs it and

sends the result back as a `tool` message; the model then writes the final answer.

Memory hooks.

- *A prompt is a steering wheel, not a program.* You change the *prefix*, and the model continues it with the most plausible text (Unit 3).
- *The model has no memory; you re-send it.* Every turn costs the whole history again.
- *Never trust, always validate.* Model output is *input* to your code; check it like user input.

Recall quiz (answers in Section 10).

1. What do the four message roles do?
2. Why does a chat that is 7 turns long cost far more than 7 times one turn?
3. Name two things `parse_json_lenient` can repair and two it cannot.
4. If one sample is right with probability 0.4, does a majority vote of 21 samples help?
5. Why is a 5-point accuracy gain on 100 test items usually not convincing?

3. Understand: the derivation

3.1 Tier 1 (easy): intuition and a picture

Unit 3 showed that an LLM does one thing: given a prefix, it gives a probability to every possible next token. **Prompt engineering is the craft of choosing the prefix.** A request such as “Translate to French: cheese” works because, in the model’s training text, that prefix is usually followed by a French word.

Four moves cover most of the craft.

1. **Say what you want, clearly** (task, format, audience, constraints). This is zero-shot prompting.
2. **Show what you want.** Add two or three worked examples (few-shot). The model copies their pattern: format, tone, labels.
3. **Give a role and rules** in the system message (“You are a support agent for ... Answer only from the provided text”).
4. **Let it think on paper.** Ask for step-by-step reasoning before the answer.

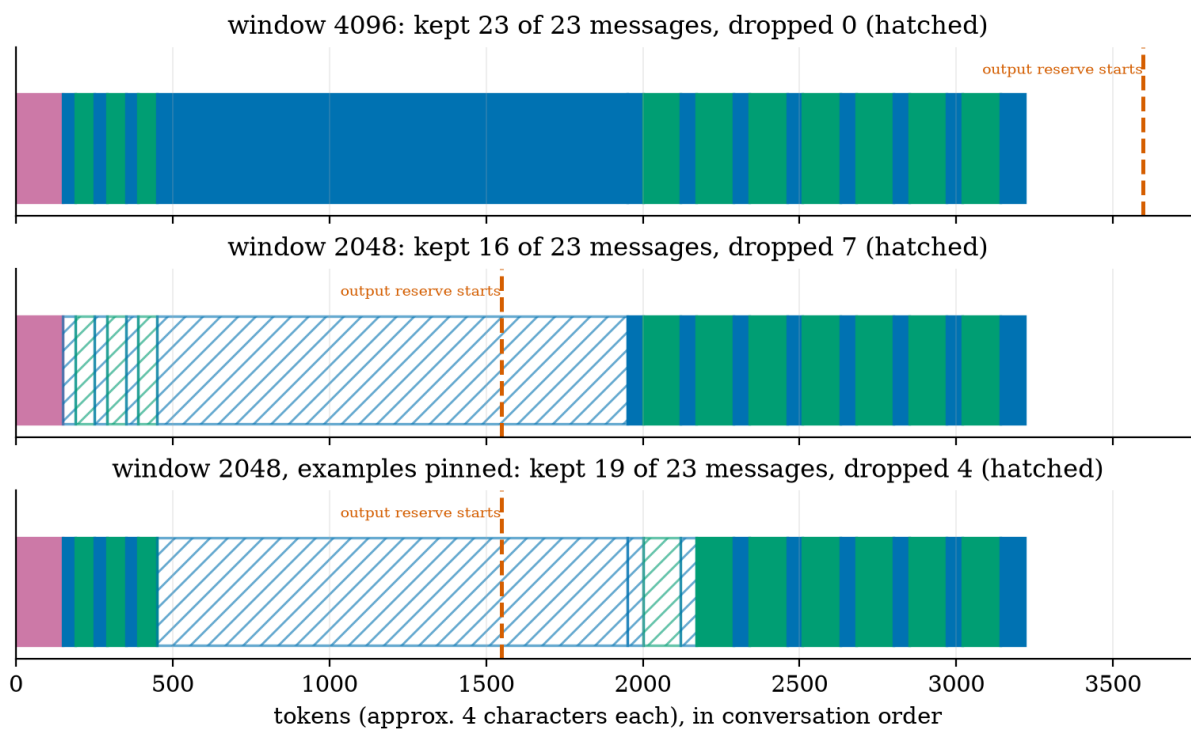
All of this text must fit in the **context window**, together with the room needed for the reply. Figure 1 packs a 23-message conversation into two window sizes.

Read Figure 1 as a warning. A simple “keep the newest messages” rule is *silent*: nothing tells the model, or you, that its instructions’ examples and the evidence for the answer are gone. The module’s `fit_context` therefore has an explicit `pin` flag for messages that must stay.

3.2 Tier 2 (medium): the definitions made precise

3.2.1 Templates

A **prompt template** is a string with placeholders that is filled for each request. The most common failure is a silent typo (a placeholder left unfilled, or a variable that never appears). The module’s `render` is deliberately strict:



A conversation drawn to scale (purple = system prompt, blue = user messages, green = assistant messages; the long blue block is retrieved documents). Top: a 4096-token window holds everything (3220 tokens plus a 500-token reserve for the reply). Middle: with a 2048-token window the oldest messages are dropped first: all three few-shot examples and the retrieved documents disappear (hatched), exactly the parts the model needs. Bottom: pinning the three examples keeps them, but the retrieved documents (1500 tokens) still cannot fit.

```
render("Summarise {{text}} in {{n}} bullet points", text=doc, n=3) # fine
render("Summarise {{text}} in {{n}} bullet points", text=doc)      # KeyError: missing
    template variables ['n']
render("Summarise {{text}}", text=doc, style="formal")              # ValueError: unused
    variables ['style']
```

A **few-shot prompt** is a template whose body is a list of messages: the system message, then each example as a (user, assistant) pair, then the real query (`few_shot_messages`).

3.2.2 Conversation cost: the model has no memory

Each call re-sends the system prompt (of s tokens) and the whole history. If every turn adds t tokens to the history (user message plus reply), over K turns the total input processed is

$$\sum_{k=1}^K (s + kt) = Ks + t \frac{K(K+1)}{2}. \quad (1)$$

(this approximates the input at turn k as $s + kt$: the $k - 1$ earlier turns plus the new message counted as a full turn). It grows **quadratically** in the number of turns. Worked check with $s = 150$, $t = 170$ (50 for the user, 120 for the reply), $K = 7$: $7 \cdot 150 + 170 \cdot 28 = 5810$ tokens processed in total, against $150 + 170 \cdot 7 = 1340$ tokens if the model remembered by itself. This is why long chats get expensive and why history must be trimmed or summarised.

3.2.3 Chain-of-thought

Wei et al. (2022) show that giving the model examples whose answers *include their reasoning steps* improves its performance on arithmetic, commonsense and symbolic tasks, and that this ability “emerges” in sufficiently large models. Their headline: a PaLM 540B model prompted with just eight chain-of-thought exemplars reached state-of-the-art accuracy on the GSM8K benchmark of math word problems. Kojima et al. (2022) found that even with *no* examples, adding “Let’s think step by step” elicits reasoning.

An intuition for why it can help (not a theorem). A Transformer with L layers performs a bounded amount of sequential computation to produce each token. A problem that needs many sequential steps cannot be solved “in the head” in one token. If the model *writes* the intermediate results, each written token becomes part of the context for the next, which gives it more sequential steps. I state this as a heuristic; why chain-of-thought works is an open research topic (Section 3.5), and Wei et al. report gains only for large models.

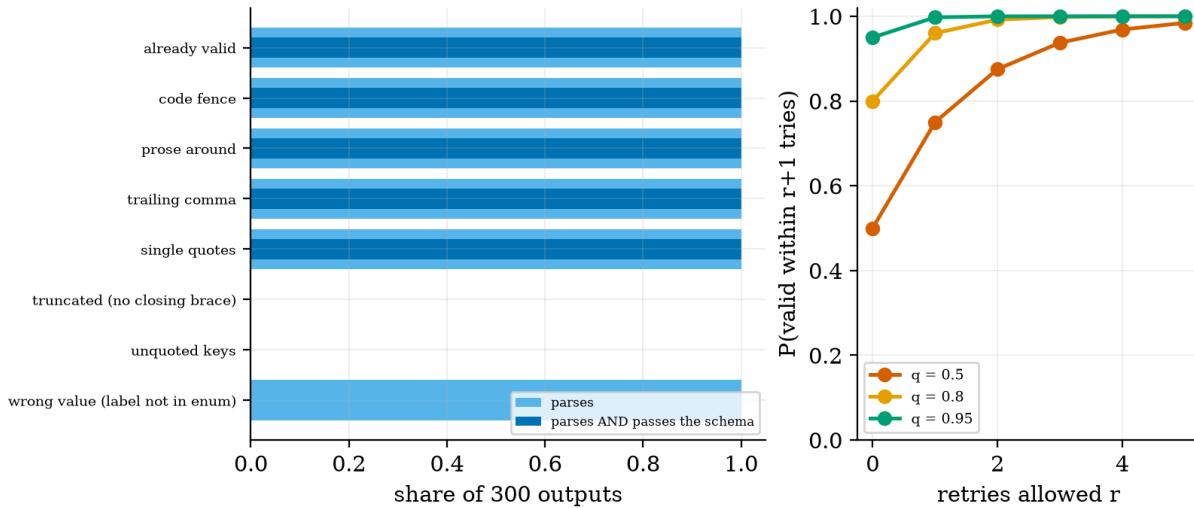
3.2.4 Structured output: validate, repair, retry

If your code must read the model’s reply, ask for JSON and **treat the reply as untrusted input**. Three layers:

1. **Validate** against a schema. The module’s `validate` supports `type`, `required`, `properties`, `enum`, `minimum`, `maximum` and `items`. Example: `{"label": "mixed"}` fails with `$.label: 'mixed' not in ['positive', 'negative', 'neutral']`.

2. **Repair** harmless formatting damage with `parse_json_lenient`: strip a code fence, cut prose around the JSON, remove a trailing comma, turn single quotes into double quotes.
3. **Retry** with feedback (`structured_call`): append the validation errors as a new user message and ask again.

Figure 2 measures layer 2 on 300 synthetic outputs per damage type, made by corrupting valid JSON (these are damage patterns I applied; how often real models produce each pattern is not measured here).



Left: share of 300 corrupted outputs that `parse_json_lenient` can parse (light blue) and that then also pass the schema (dark blue). Code fences, surrounding prose, trailing commas and single quotes are repaired every time. Truncated JSON and unquoted keys are never repaired (0%). A syntactically valid but wrong value (a label outside the allowed set) parses (100%) but fails the schema (0%), so validation is a separate step. Right: probability of a valid answer within $r + 1$ attempts, $1 - (1 - q)^{r+1}$, if each attempt is valid with probability q independently.

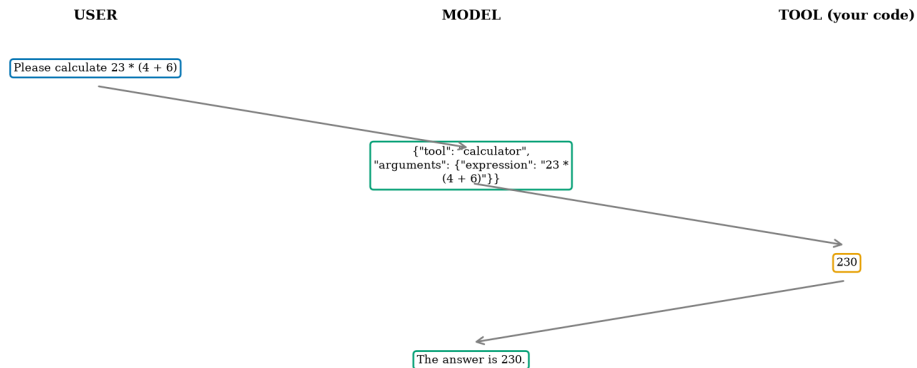
The right-hand curve shows why retries are cheap *if attempts are independent*: with $q = 0.5$ per attempt, three retries give $1 - 0.5^4 = 0.94$. When the model fails for a systematic reason (the schema is ambiguous), attempts are not independent and the curve is too optimistic.

3.2.5 Tool calling

The model cannot run code or read your database; it can only *write text*. Tool calling turns text into action under your control:

1. you describe each tool to the model (a name, a description, a JSON schema for its arguments);
2. the model replies with a tool request in a fixed format;
3. **your code** checks the arguments against the schema, runs the function, and catches any error;
4. the result goes back as a `tool` message and the model writes the final answer.

The module's `ToolRegistry.call` returns errors as *text* (`error: unknown tool 'x', error: $: missing required key 'expression'`) so a bad request does not crash the loop and the model can correct itself. The calculator tool is a lesson in itself: while writing it I first allowed any string made of digits and the



The message flow of one tool call, drawn from the real log of the module’s `run_tool_loop` with a mock model: the user’s request, the model’s JSON tool request, the tool result 230 computed by your code, and the final answer.

operators, which still accepts `9**9**9`, a computation that never finishes. The final `safe_calc` rejects `**` and long inputs, and a whitelist plus `eval` is still only a teaching shortcut; production code should use a real expression parser.

3.3 Tier 3 (medium-hard): the main derivations

3.3.1 Self-consistency: when does voting help?

Wang et al. (2022) sample several reasoning paths at a non-zero temperature (Unit 3) and take the **majority answer**; their Table 1 compares aggregation rules on PaLM 540B. Under a simple model we can compute exactly when this helps. Suppose each sample is correct with probability p , independently, and wrong answers are spread over many different values, so they do not combine into a competing majority. With an odd number k of samples the majority is correct when more than $k/2$ are correct:

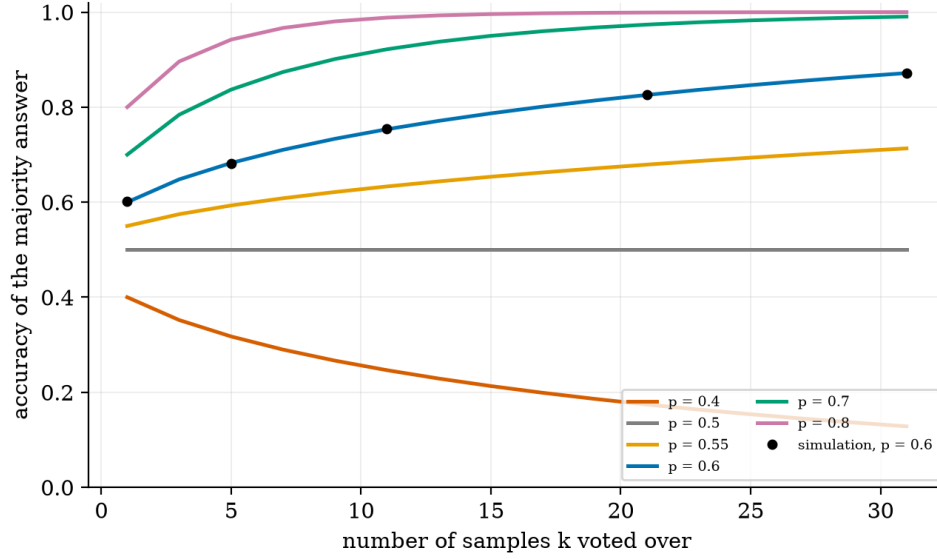
$$A_k(p) = \sum_{j=\lceil (k+1)/2 \rceil}^k \binom{k}{j} p^j (1-p)^{k-j}. \quad (2)$$

For $k = 3$: $A_3(p) = p^3 + 3p^2(1-p) = 3p^2 - 2p^3$, and

$$\begin{aligned} A_3(p) - p &= 3p^2 - 2p^3 - p = -p(2p^2 - 3p + 1) \\ &= -p(2p-1)(p-1) = p(1-p)(2p-1). \quad (\text{factor the quadratic}) \end{aligned} \quad (3)$$

Since $p(1-p) > 0$ for $0 < p < 1$, voting helps exactly when $2p-1 > 0$, that is $p > \frac{1}{2}$; it hurts when $p < \frac{1}{2}$ and does nothing at $p = \frac{1}{2}$. Worked check: $p = 0.7$ gives $A_3 = 3(0.49) - 2(0.343) = 0.784 > 0.7$. The same holds for every odd k , because $A_k(p) + A_k(1-p) = 1$ (the majority is correct or wrong, and swapping $p \leftrightarrow 1-p$ swaps the two).

What the assumption hides. Real samples from one model are *not* independent: if the model has a misconception, it repeats it, and the wrong answers can agree. Then the true majority accuracy is lower than (2), and the gain from adding samples stops early. Equation (2) is an upper-bound intuition, not a forecast.



Accuracy of the majority answer against the number of samples k , for several single-sample accuracies p (exact, from (2)), with a simulation of 200{,}000 trials for $p = 0.6$ (black dots). At $p = 0.6$: 0.683 at $k = 5$, 0.753 at $k = 11$, 0.826 at $k = 21$, 0.872 at $k = 31$; the simulation gives 0.601, 0.682, 0.754, 0.825, 0.872. At $p = 0.5$ nothing changes; at $p = 0.4$ voting makes it worse (0.317, 0.247, 0.174, 0.128).

3.3.2 Comparing two prompts: paired data and sample size

You have two prompts A and B and a test set of n items. Score each item 1 (correct) or 0. Because both prompts see the *same* items, compare them item by item. Let $d_i = \text{score}_B(i) - \text{score}_A(i) \in \{-1, 0, +1\}$. Let b be the count of items where only B is right and c where only A is right. Then

$$\bar{d} = \frac{b - c}{n}, \quad \widehat{\text{Var}}(d_i) = \frac{b + c}{n} - \bar{d}^2, \quad \text{SE}(\bar{d}) = \sqrt{\frac{\widehat{\text{Var}}(d_i)}{n}}, \quad (4)$$

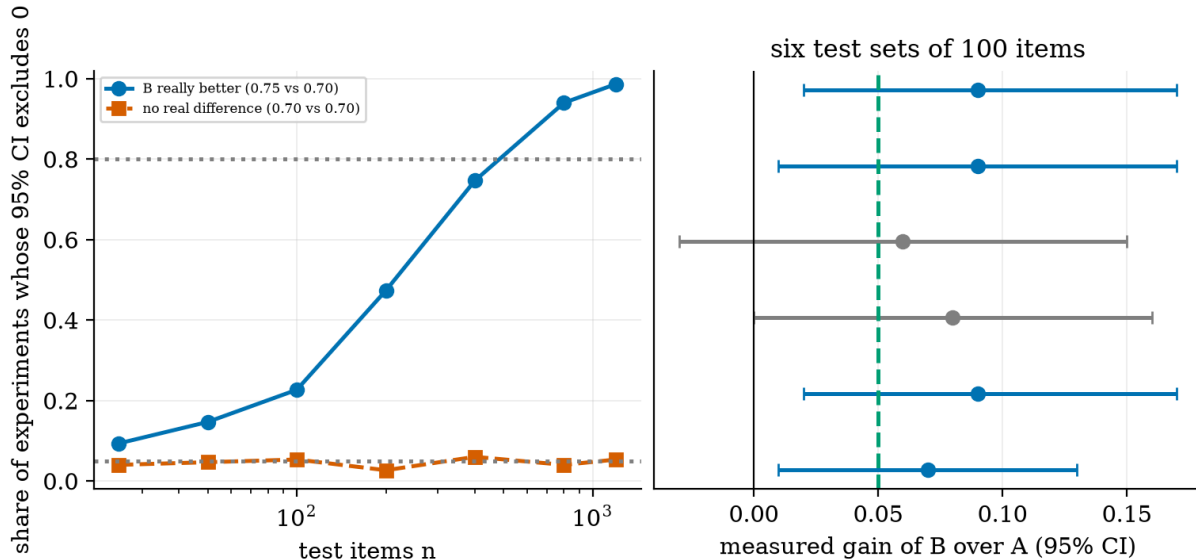
(the first term of the variance is the share of items where the prompts disagree, since $d_i^2 = 1$ exactly there), and an approximate 95% confidence interval is $\bar{d} \pm 1.96 \text{SE}$. **Worked example:** $n = 100$, $b = 15$, $c = 8$. Then $\bar{d} = 0.07$, $\widehat{\text{Var}} = 0.23 - 0.0049 = 0.2251$, $\text{SE} = \sqrt{0.2251/100} = 0.0474$, and the interval is $0.07 \pm 0.093 = [-0.023, 0.163]$. It contains 0: **a 7-point gain on 100 items is not distinguishable from noise.**

How many items are needed? To detect a true gain δ with 80% power at the 5% level, the standard approximation is

$$n \approx \frac{(1.96 + 0.84)^2 \text{Var}(d_i)}{\delta^2}. \quad (5)$$

In the simulation below the two prompts disagree on 15.5% of the items (variance 0.152) and the true gain is $\delta = 0.05$, so $n \approx 7.84 \cdot 0.152 / 0.0025 = 478$. The **bootstrap** gives the same kind of interval without formulas: resample the n values d_i with replacement many times and take the 2.5% and 97.5% quantiles of the resampled means (`paired_bootstrap_ci`).

Figure 5 agrees with (5): 80% power needs roughly 450 to 500 items for a 5-point gain.



A simulation (not real prompts): outcomes for two prompts with true accuracies 0.70 and 0.75, which agree on most items. Left: the share of experiments in which the 95% bootstrap interval excludes zero, against the number of test items (150 simulated experiments per point). With a real gain the share is 9% at $n = 25$, 23% at $n = 100$, 47% at $n = 200$, 75% at $n = 400$, 94% at $n = 800$. With no real gain it stays near 5% (0.03 to 0.06), as it should. Right: six simulated test sets of 100 items: two of the six intervals include zero (grey).

3.3.3 The winner’s curse

If you run many small experiments and report only the significant ones, the reported gains are inflated. In the same simulation at $n = 100$ the average measured gain over all test sets is 0.0501 (true gain 0.05, standard deviation 0.039): the estimator is unbiased. But only 22.5% of the experiments were significant, and among *those* the average measured gain was 0.100, **twice the true effect**. Selecting on significance selects lucky samples. The practical rule: fix the test set and the metric *before* trying prompt variants, and be sceptical of a large gain found on a small set.

3.4 Tier 4 (hard): robustness and risk

3.4.1 Long contexts

Attention cost grows with context length (Unit 2), but there is a second issue: models do not use all positions equally. Liu et al. (2023) found a **U-shaped** curve: models use relevant information best at the very beginning or end of the input, and performance degrades when it sits in the middle, even for models built for long contexts. A practical consequence for Figure 1: put the instructions and the question at the edges, and do not assume that more retrieved text is always better.

3.4.2 Constrained output

Instead of repairing bad JSON afterwards, a serving system can prevent it: at every step, set the logit of every token that would break the grammar or schema to $-\infty$ *before* the softmax of Unit 3,

$$p_i = \frac{e^{z_i/T} \mathbf{1}[i \text{ allowed}]}{\sum_j e^{z_j/T} \mathbf{1}[j \text{ allowed}]},$$

the same mechanism as top- k filtering, with the allowed set decided by the schema instead of by rank. This guarantees *syntax*; it cannot guarantee that the *content* is true.

3.4.3 Prompt injection (preview)

Your prompt contains text you control (the system message) and text you do not (the user’s message, a web page, a retrieved document). A model cannot reliably tell the two apart: a retrieved page that says “ignore previous instructions and ...” is just more text in the prefix. Practical defences (Unit 9 treats them properly): keep secrets and permissions out of the prompt, validate every tool call as in Section 3.2.5, give tools the least privilege, and never run model output as code.

3.5 Tier 5 (hardest): open questions

1. **Why does chain-of-thought work, and are the written steps the model’s real reasoning?** The text of a chain of thought may be a plausible story rather than a record of the computation. Whether the steps are *faithful* to what the model actually did is an active research question; do not use a model’s explanation as evidence that its answer is right.
2. **Prompt sensitivity.** Small changes in wording, example order or formatting can change measured accuracy. How to report a result so it survives such changes is unsettled; the minimum is to test several prompt variants on a fixed, large test set (Section 3.3.2).
3. **When is a prompt better than fine-tuning?** There is no general rule. Units 5 and 6 give the trade-offs (retrieval, fine-tuning).

4. Apply

4.1 Worked problems

Problem 1 (template). `render("Hi {{name}}", nam="Ann")` raises: the variable is called `name`, not `nam`. Silent failure is avoided because both the missing and the unused name are reported.

Problem 2 (context budget). A window of 4096 tokens, a reserved reply of 500, a system prompt of 150, three examples of 100 tokens each and a new question of 80. Free room for history and retrieved text: $4096 - 500 - 150 - 300 - 80 = 3066$ tokens.

Problem 3 (conversation cost). A bot with a 150-token system prompt, 170 tokens per turn, 7 turns: 5810 input tokens in total by (1), against 1340 if only new text counted, a factor of 4.3.

Problem 4 (voting). $p = 0.7$, $k = 3$: $A_3 = 0.784$. With $k = 5$: $\sum_{j=3}^5 \binom{5}{j} p^j (1-p)^{5-j} = 0.837$ (the table of Figure 4 gives 0.837).

Problem 5 (is B better?). $n = 100$, $b = 15$, $c = 8$: the 95% interval is $[-0.023, 0.163]$, so no. If the same proportions held with $n = 400$ ($b = 60$, $c = 32$): $\bar{d} = 0.07$, $\widehat{\text{Var}} = 0.23 - 0.0049 = 0.2251$, $\text{SE} = \sqrt{0.2251/400} = 0.0237$, interval $[0.024, 0.116]$: now it excludes zero.

4.2 Code and figures

Everything is in `codes/llm/prompting_tools/` (see its README). The three syllabus projects fit the same skeleton, with the model hidden behind one method so you can swap the mock for a real one:

```

class MyLLM:
    def complete(self, messages): ...
    # any object with complete(messages) -> str
    # call your provider here

# 1. domain chatbot: system prompt + few-shot examples + fit_context to keep the history in
#    bounds
msgs = P.few_shot_messages(system, examples, user_question)
kept, dropped = P.fit_context(msgs, window=4096, reserve_output=500)

# 2. document summariser: template + structured output
obj, attempts = P.structured_call(llm, msgs, {"type": "object", "required": ["summary", "
    bullets"], ...})

# 3. assistant with tools: registry + validated loop
answer, log = P.run_tool_loop(llm, registry, "Please calculate 23 * (4 + 6)")

```

With the bundled `MockLLM` the loop returns “The answer is 230.” and the log of Figure 5. The mock only tests the plumbing: it pattern-matches arithmetic and understands nothing. The Colab notebook `u4_prompting_real_model.ipynb` replaces it with a small instruction-tuned model from the Hugging Face hub (it was **not executed** while writing this document).

Try it in the playground

The **Context window budget** widget lets you set the window and the size of each part of a conversation and see what is dropped and what the next turn costs. The **Majority vote** widget plots the accuracy of voting for any single-sample accuracy p and number of samples k , next to a simulation.

5. Analyze

Checklist items used: limits (5.1), dominance (5.2), sensitivity (5.3), failure modes (5.4), constructed-versus-measured (5.5).

5.1 Limits

- $k = 1$: $A_1(p) = p$. $p = \frac{1}{2}$: $A_k = \frac{1}{2}$ for every odd k (Figure 4). $p \rightarrow 1$: $A_k \rightarrow 1$ quickly ($p = 0.8$, $k = 11$ gives 0.988).
- Conversation of one turn: (1) reduces to $s + t$.
- A pinned message that is too large: `fit_context` raises `ValueError` instead of silently truncating.

5.2 Which term dominates

In (1) the history term $tK(K+1)/2$ overtakes the system-prompt term Ks once $t(K+1)/2 > s$; with $s = 150$ and $t = 170$ that is already true from the first turn ($K = 1$: $170 > 150$). So for chats, history, not the system prompt, dominates the cost.

5.3 Sensitivity

Majority accuracy is most sensitive to p near $\frac{1}{2}$: at $k = 21$, $A_k(0.55) = 0.679$ but $A_k(0.6) = 0.826$. A small improvement in single-sample accuracy is amplified when voting. Evaluation power is most sensitive to the disagreement rate: if two prompts disagree on only 5% of the items, $\text{Var}(d_i) \approx 0.05$ and (5) asks for about 157 items; at 30% disagreement it asks for 941.

5.4 Failure modes

- Silently dropped instructions or evidence (Figure 1).
- Syntax repaired but content wrong (Figure 2, last row).
- Tool requests that are valid JSON but dangerous (9**9**9).
- Voting with dependent samples (Section 3.3.1).
- Declaring a winner from a small set (Section 3.3.2), or from the significant subset only (Section 3.3.3).

5.5 Constructed versus measured

Figures 2 and 5 are **constructed**: the damage patterns and the prompt outcomes were generated by code, to show the mathematics. Figure 4 is exact mathematics (with a simulation check). Figures 1 and 3 use made-up message sizes and a mock model. Nothing in this session measures a real model, and the claims about real models (chain-of-thought gains, the U-shaped context curve) are cited, not reproduced.

6. Evaluate

Prompting versus the alternatives. Prompting needs no training and is easy to change, but is limited by the context window and by what the model already knows. Retrieval (Unit 5) adds knowledge; fine-tuning (Unit 6) changes behaviour permanently. Prompting is the right first thing to try; it is the wrong tool when the knowledge changes daily (use retrieval) or when you need a consistent style across millions of calls (consider fine-tuning).

Chain-of-thought and voting: cost. Both multiply the tokens you pay for: chain-of-thought lengthens each answer; voting over k samples costs k times as much. Figure 4 shows what the extra cost buys *under independence*; measure on your task.

Structured output. Constrained decoding is the strongest guarantee of syntax; validation with retry is the portable fallback that works with any provider.

When not to trust a prompt result. A gain on fewer than a few hundred items; a result found by trying many variants on one test set; a prompt whose success depends on one lucky example order.

Wider context. Prompt engineering is partly empirical craft and partly software engineering: versioned templates, tests, validation and logs. The durable skills are the ones in this session (budgeting context, validating outputs, measuring honestly); specific phrasings come and go with each model release.

7. Create

What you can build from this

1. **Domain chatbot:** a system prompt, three few-shot examples and `fit_context` with pinned examples; show with a long fake conversation that the examples survive.
2. **Document summariser:** a template, a JSON schema (`summary`, `bullets`, `confidence`), and `structured_call`; list the failure cases your validator catches.
3. **Tool-using assistant:** add a second tool (a date function or a lookup in a small dictionary) and make the registry reject calls with wrong types.
4. **Prompt A/B harness:** a function that takes two prompts, a test set and a model, runs both, and prints the paired difference with a bootstrap interval and the number of items still needed.
5. **Combine with Unit 3:** use the sampler from Unit 3 to generate k samples at a temperature, then take the majority answer; plot accuracy against k .
6. **Module for the open-source package:** add a JSON-schema oneOf and pattern check to validate, with tests.

8. Visualization lesson

charts taught here. How to read a power curve: find the point where it crosses 0.8 on the y-axis and read the number of items on the x-axis; the second curve shows the false-positive rate, which should sit at 0.05.

Chart type	Best for	How it misleads	matplotlib call
to-scale stacked bar (a “budget bar”)	parts of a limited resource (Fig. 1)	looks exact when token counts are only estimates	<code>ax.barh(0, w, left=x)</code> in a loop
line plot with simulation overlay	checking a formula against a simulation (Fig. 4)	few simulation points hide noise; show the trial count	<code>ax.plot(k, exact); ax</code> <code>.plot(kk, mc, "ko")</code>
interval (error-bar) plot	an estimate with uncertainty (Fig. 5 right)	the interval width depends on the method (bootstrap, normal)	<code>ax.errorbar(d, i,</code> <code>xerr=[[d-lo],[hi-d]],</code> <code>fmt="o")</code>
power curve	sample size planning (Fig. 5 left)	assumes the simulated effect and disagreement rate	<code>ax.semilogx(n, share,</code> <code>"o-")</code>
message-flow diagram	who sends what to whom (Fig. 3)	looks like a log but is hand-laid	<code>ax.annotate("", xy,</code> <code>xytext, arrowprops</code> <code>=...)</code>

9. Problem set

Graded exercises

1. (easy) Write a few-shot prompt (three examples) that classifies a review as positive or negative, and say which of the four message roles each part belongs to.
2. (medium) A window of 8192 tokens, a reserve of 1000 for the reply and a system prompt of 400. Each turn adds 250 tokens. How many full turns fit before the oldest must be dropped?
3. (medium-hard) Prove that $A_k(p) + A_k(1 - p) = 1$ for odd k and use it to show $A_k(\frac{1}{2}) = \frac{1}{2}$.
4. (hard) Two prompts have $n = 250$, $b = 30$, $c = 14$. Compute $\bar{d}$, SE and the 95% interval with (4). Does it exclude zero? What n would a 80%-power test of a true gain of 0.03 need if the disagreement rate is 0.18?
5. (hardest) In Section 3.3.3 the average measured gain among significant experiments was 0.100 for a true gain of 0.05. Explain why this happens, then explain why it would not happen if you reported *all* experiments, and say what this implies for a team that tries 20 prompts on the same 100 test items and reports the best.

10. Retrieval practice and recap

Answers to the Section 2 quiz.

1. System: standing rules. User: the request. Assistant: the model's earlier replies. Tool: results of functions your code ran.
2. The model is stateless, so each turn re-sends the whole history: the cost grows like $K(K + 1)/2$ (Equation (1)), not like K .
3. Repairs: code fences, surrounding prose, trailing commas, single quotes. Cannot repair: truncated JSON, unquoted keys; and it cannot fix a wrong *value* that is valid JSON (Figure 2).
4. No: with $p = 0.4 < \frac{1}{2}$ the majority gets worse (0.174 at $k = 21$).
5. With 100 items the 95% interval for a 5-point gain is roughly ± 0.09 wide, so it includes zero; about 450 to 500 items are needed (Figure 5).

Explain it to a friend.

- Why does the same chatbot cost more for the 10th question than for the 1st?
- Why must code treat a model's JSON like untrusted user input?
- Why can “7 points better on 100 questions” still be noise?

Re-derive tomorrow without looking: (a) $A_3(p) - p = p(1 - p)(2p - 1)$; (b) the cost (1); (c) the paired interval (4) for $n = 100$, $b = 15$, $c = 8$.

11. Where next

- **Next:** Unit 5 (retrieval-augmented generation): the retrieved documents of Figure 1 are what RAG supplies, and the evaluation tools of Section 3.3 are how you will judge it.

- **Terms met but not taught:** constrained decoding grammars, prompt caching, LLM-as-judge (Unit 9).
- **Open threads:** whether written reasoning steps are faithful (Section 3.5); how to report prompt results so they survive wording changes.